chapter 14, but for the time being, it will suffice to say that this property is used to determine whether the application is currently running in Development mode, and if so, the `UseException- Handler` method is called, which adds middleware for catching errors and displaying their details in the browser. Otherwise, the error page you saw in the Pages folder is used to display a bland message that conceals any sensitive information about the specifics of the error to the user, such as database connection strings containing user credentials or information about file paths on the server. Middleware that adds an HTTP Strict Transport Security header is also registered (`app.UseHsts()`) but only if the application is not running in Development mode. This header tells the browser to only use HTTPS when accessing the website. I cover this in more detail in chapter 13.

The `UseHttpsRedirection` method adds middleware that ensures that any HTTP requests are redirected to HTTPS. Then, after this, the static files middleware is registered. By default, an ASP.NET Core application doesn't support serving static files, such as images, stylesheets, and script files. You have to opt in for this feature, and you do so by adding the static files middleware. This middleware configures the wwwroot folder to allow static files to be requested directly and serves them to the client.

Routing middleware is responsible for selecting which *endpoint* should be executed, based on information included in the request. I discuss how routing works in Razor Pages in chapter 4. Then, authorization middleware is registered, which is responsible for determining if the current user is authorized to access the resource that is being requested. Authorization is covered in chapter 10.

Finally, the `MapRazorPages` method adds middleware to the pipeline that initially configures Razor Pages as endpoints. This middleware is also